We present a step-by-step testing algorithm; excerpts of the processing Python code and an example dataset are provided at the end of this section.

1. First, consecutive frames were acquired and then stored in an array. Load the signal and perform pre-processing
2. Background was removed by estimating the background as the mean of the first 1,000 frames and subtracting it from each frame. Extract instantaneous phase using the Hilbert transform
3. Frame intensities were then normalized, denoised, and contrast-enhanced.
4. A region of interest (ROI (10*10 pixels)) was selected on the image, and its mean intensity was computed.
5. An FFT was applied to the resulting 1D signal; components outside 0.01–0.15 cycles/frame were zeroed, and the signal was reconstructed via inverse FFT to suppress unwanted fluctuations (e.g., vibration and …).
6. The Hilbert transform was applied to obtain the instantaneous phase.
7. Using this phase, points corresponding to four consecutive steps (0, π/2, π, 3π/2) were identified and plotted.

--- Imports (required libraries) ---

```
import matplotlib
matplotlib.use('TkAgg')
from matplotlib import pyplot as plt
import cv2
import numpy as np
from scipy.signal import find_peaks
from scipy.signal import hilbert
```

1. First, consecutive frames were acquired and then stored in an array. Load the signal and perform pre-processing

```
def load_frames(folder_path):
    files = sorted([f for f in os.listdir(folder_path) if
f.endswith('.png')])
    frames = [normalize(cv2.imread(os.path.join(folder_path, f),
cv2.IMREAD_GRAYSCALE)[100:858,100:856]) for f in files]

    return np.array(frames)

path1=r'C:\Users\mojtaba\Desktop\image_prosess\path_t3'
frames = load_frames(path1)
```

2-Background was removed by estimating the background as the mean of the first 1,000 frames and subtracting it from each frame. Extract instantaneous phase using the Hilbert transform.

```
background = np.mean(frames[0:1000], axis=0)
frames_no_bg = frames - background
```

3-Frame intensities were then normalized, denoised, and contrast-enhanced.

```
img_clahe_np=[]
def normalize(img):
    return cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
for i,ff in enumerate(frames_no_bg):
    # Step 1: Denoising with Non-Local Means
    j=i+1600
    nn=normalize(ff)
    img_denoised = cv2.fastNlMeansDenoising(nn, h=10, templateWindowSize=7,
searchWindowSize=21)
    # Step 2: Contrast Enhancement with CLAHE
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    img_clahe = clahe.apply(img_denoised)
    img_clahe_np.append(img_clahe)
img_clahe_np=np.array(img_clahe_np)
```

4-A region of interest (ROI (10*10 pixels)) was selected on the image, and its mean intensity was computed.

```
image_fillter=[]
for ff in clean_frame:
    roi = ff[y1:y2, x1:x2]
    image_fillter.append(np.mean(roi))
image_fillter=np.array(image_fillter)
np.save('clean_frame.npy',image_fillter)
```

5-An FFT was applied to the resulting 1D signal; components outside 0.01–0.15 cycles/frame were zeroed, and the signal was reconstructed via inverse FFT to suppress unwanted fluctuations (e.g., vibration).

```
filtered_signal = np.load('low_pass_signal.npy').flatten()
fft_signal = np.fft.fft(filtered_signal)
freqs = np.fft.fftfreq(len(filtered_signal), d=1)
low_cutoff = 0.01
high_cutoff = 0.15
fft_bandpass = fft_signal.copy()
fft_bandpass[np.abs(freqs) < low_cutoff] = 0
fft_bandpass[np.abs(freqs) > high_cutoff] = 0
bandpassed_signal = np.fft.ifft(fft_bandpass).real
```

6-The Hilbert transform was applied to obtain the instantaneous phase.

```
# Apply the Hilbert transform to the band-passed signal
analytic_signal = hilbert(bandpassed_signal)
# Instantaneous phase in the range [-π, π]
instantaneous_phase = np.angle(analytic_signal) # phase in [-π, π]
```

7-Using this phase, points corresponding to four consecutive steps (0, π/2, π, 3π/2) were identified and plotted.

```python
def extract_phase_points(phase_array, signal, target_phase, label, atol=0.05,
min_distance=10):
    """
     From all samples whose instantaneous phase is close to `target_phase`,
     keep only one representative index per contiguous group by enforcing a
     minimum frame spacing (`min_distance`).

     Parameters
    ----------
    phase_array : np.ndarray
        Instantaneous phase values in radians, typically in [-π, π].
    signal : np.ndarray
        The (band-passed) intensity trace; not used directly here but kept
         for possible future gating (e.g., amplitude SNR checks).
    target_phase : float
        Target phase in radians (e.g., 0, π/2, π, 3π/2).
    label : str
        A human-readable label for the phase (e.g., '0', 'π/2', 'π', '3π/2').
Used to apply special handling near ±π due to phase wrapping.
    atol : float, default=0.05
        Absolute tolerance (in radians) when matching `phase_array` to
`target_phase`.
    min_distance : int, default=10
        Minimum separation (in frames) between kept indices to avoid
oversampling within the same cycle.
    """
    # Candidate indices whose instantaneous phase is within `atol` of the
target
    indices = np.where(np.isclose(phase_array, target_phase, atol=atol))[0]
    # Special handling for π (or -π): account for wrapping near ±π
    if label in ['π', '-π']:
        indices = np.where((phase_array > 3.0) | (phase_array < -3.0))[0]
    # Enforce a minimum spacing to keep only one representative per cycle
    selected = []
    prev = -min_distance
    for idx in indices:
        if idx - prev >= min_distance:
            selected.append(idx)
            prev = idx
    return np.array(selected)

# --- Target phases and their plotting styles ---
phase_targets = {'0': 0, 'π/2': np.pi/2, 'π': np.pi, '3π/2': -np.pi/2}
phase_colors = {'0': 'k', 'π/2': 'r', 'π': 'b', '3π/2': 'g'}
phase_markers = {'0': 'x', 'π/2': '^', 'π': 'o', '3π/2': 's'}
# --- Extract representative indices for each phase ---
phase_points = {}
for label, value in phase_targets.items():
    points = extract_phase_points(instantaneous_phase, bandpassed_signal,
value, label)
    phase_points[label] = points
# --- Plot the band-passed trace and overlay the phase picks ---
plt.figure(figsize=(15, 5))
plt.plot(bandpassed_signal, label='Bandpassed Signal', linewidth=1.5)
for label in phase_targets:
    if label == 'π':
        # Identify valleys to stabilize π selection (where the cosine
```

```
component flips sign)
        valleys, _ = find_peaks(-bandpassed_signal, distance=10)
        # Select only valleys that fall inside the π (or -π) region per
Hilbert phase
        pi_region = (instantaneous_phase > 2.8) | (instantaneous_phase < -
2.8)
        pts = [v for v in valleys if pi_region[v]]
    else:
        # Use the phase-matched indices for 0, π/2, and 3π/2
        pts = phase_points[label]
    plt.plot(pts, bandpassed_signal[pts], linestyle='None',
marker=phase_markers[label], color=phase_colors[label], label=f'Phase ≈
{label}')
plt.title("Bandpassed Signal with Representative Phase Points")
plt.xlabel("Frame Index")
plt.ylabel("Intensity")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

--- Imports (required libraries) ---

```
import matplotlib matplotlib.use('TkAgg') from matplotlib import pyplot as plt
import cv2 import numpy as np from scipy.signal import find_peaks from
scipy.signal import hilbert
```

1. First, consecutive frames were acquired and then stored in an array. Load the signal and perform pre-processing

```
def load_frames(folder_path): files = sorted([f for f in os.listdir(folder_path) if
f.endswith('.png')]) frames = [cv2.imread(os.path.join(folder_path, f), cv2.IMREAD_GRAYSCALE) for
f in files] return np.array(frames) frames =
load_frames(r'C:\Users\mojtaba\Desktop\image_prosess\path_t3')
```

2-Background was removed by estimating the background as the mean of the first 1,000 frames and subtracting it from each frame. Extract instantaneous phase using the Hilbert transform

```
background = np.mean(frames[0:1000], axis=0) frames_no_bg = frames - background
```

3-Frame intensities were then normalized, denoised, and contrast-enhanced.

```
def normalize(img): return cv2.normalize(img, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
for i,ff in enumerate(frames_no_bg): # Step 1: Denoising with Non-Local Means j=i+1600
nn=normalize(ff) img_denoised = cv2.fastNlMeansDenoising(nn, h=10,
templateWindowSize=7, searchWindowSize=21) # Step 2: Contrast Enhancement with CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)) img_clahe =
clahe.apply(img_denoised) img_clahe_np.append(img_clahe)
img_clahe_np=np.array(img_clahe_np)
```

4-A region of interest (ROI (10*10 pixels)) was selected on the image, and its mean intensity was computed.

```
image_fillter=[] for ff in clean_frame:
```

```
roi = ff[y1:y2, x1:x2] image_fillter.append(np.mean(roi))
image_fillter=np.array(image_fillter)
np.save('clean_frame.npy',image_fillter)
```

5-An FFT was applied to the resulting 1D signal; components outside 0.01–0.15 cycles/frame were zeroed, and the signal was reconstructed via inverse FFT to suppress unwanted fluctuations (e.g., vibration).

```
filtered_signal = np.load('low_pass_signal.npy').flatten() # fft_signal =
np.fft.fft(filtered_signal) freqs = np.fft.fftfreq(len(filtered_signal), d=1)
low_cutoff = 0.01 high_cutoff = 0.15 fft_bandpass = fft_signal.copy()
fft_bandpass[np.abs(freqs) < low_cutoff] = 0 fft_bandpass[np.abs(freqs) >
high_cutoff] = 0 bandpassed_signal = np.fft.ifft(fft_bandpass).real
```

6-The Hilbert transform was applied to obtain the instantaneous phase.

```
# Apply the Hilbert transform to the band-passed signal analytic_signal =
hilbert(bandpassed_signal) # Instantaneous phase in the range [-π, π]
instantaneous_phase = np.angle(analytic_signal) # phase in [-π, π]
```

7-Using this phase, points corresponding to four consecutive steps (0, π/2, π, 3π/2) were identified and plotted.

```
# --- Helper to pick one representative frame per cycle near a target phase ---
def extract_phase_points(phase_array, signal, target_phase, label, atol=0.05,
min_distance=10): """ From all samples whose instantaneous phase is close to
`target_phase`, keep only one representative index per contiguous group by
enforcing a minimum frame spacing (`min_distance`). Parameters ----------
phase_array : np.ndarray
```

Instantaneous phase values in radians, typically in [-π, π]. signal : np.ndarray The (band-passed) intensity trace; not used directly here but kept for possible future gating (e.g., amplitude SNR checks). target_phase : float Target phase in radians (e.g., 0, π/2, π, 3π/2). label : str A human-readable label for the phase (e.g., '0', 'π/2', 'π', '3π/2'). Used to apply special handling near ±π due to phase wrapping. atol : float, default=0.05 Absolute tolerance (in radians) when matching `phase_array` to `target_phase`. min_distance : int, default=10 Minimum separation (in frames) between kept indices to avoid oversampling within the same cycle. """ # Candidate indices whose instantaneous phase is within `atol` of the target indices = np.where(np.isclose(phase_array, target_phase, atol=atol))[0] # Special handling for π (or -π): account for wrapping near ±π if label in ['π', '-π']: indices = np.where((phase_array > 3.0) | (phase_array < -3.0))[0] # Enforce a minimum spacing to keep only one representative per cycle selected = [] prev = -min_distance for idx in indices: if idx - prev >= min_distance: selected.append(idx) prev = idx return np.array(selected) # --- Target phases and their plotting styles --- phase_targets = {'0': 0, 'π/2': np.pi/2, 'π': np.pi, '3π/2': -np.pi/2} phase_colors = {'0': 'k', 'π/2': 'r', 'π': 'b', '3π/2': 'g'} phase_markers = {'0': 'x', 'π/2': '^', 'π': 'o', '3π/2': 's'} # --- Extract representative indices for each phase --- phase_points = {} for label, value in phase_targets.items(): points = extract_phase_points(instantaneous_phase, bandpassed_signal, value, label) phase_points[label] = points # --- Plot the band-passed trace and overlay the phase picks --- plt.figure(figsize=(15, 5)) plt.plot(bandpassed_signal, label='Bandpassed Signal', linewidth=1.5) for label in phase_targets: if label == 'π': # Identify valleys to stabilize π selection (where the cosine component flips sign) valleys, _ = find_peaks(-bandpassed_signal, distance=10) # Select only valleys that fall inside the π (or -π) region per

```
Hilbert phase pi_region = (instantaneous_phase > 2.8) | (instantaneous_phase < -
2.8) pts = [v for v in valleys if pi_region[v]] else: # Use the phase-matched
indices for 0, π/2, and 3π/2 pts = phase_points[label] plt.plot(pts,
bandpassed_signal[pts], linestyle='None', marker=phase_markers[label],
color=phase_colors[label], label=f'Phase ≈ {label}') plt.title("Bandpassed Signal
with Representative Phase Points") plt.xlabel("Frame Index")
plt.ylabel("Intensity") plt.legend() plt.grid(True) plt.tight_layout() plt.show()
```

4. For one representative en-face onion/tissue image, report SNR or CNR (define regions and method). This can be done from existing data and will ground the qualitative demonstration in a simple quantitative measure.

For a representative en-face tissue image, we quantified image quality by calculating the signal-to-noise ratio (SNR) and contrast-to-noise ratio (CNR). Figure 4 shows the selected image, with the signal and background regions of interest (ROIs) indicated.

The SNR and CNR were calculated according to the following definitions

$\mu_{signal}=1N\Sigma I_{signal,iNi=1}$ $\mu_{background}=1M\Sigma I_{background,jMj=1}$ $SNR=\mu_{signal}\sigma_{background}$

$CNR=\mu_{signal}-\mu_{background}\sqrt{\sigma_{signal2}+\sigma_{background2}}$